



Nathan Cartmell

Backend Developer

Guildford, UK

Email (me@ncartmell.co.uk) [GitHub](https://github.com/ncartmell) (github.com/ncartmell)

[LinkedIn](https://www.linkedin.com/in/nathan-cartmell) ([linkedin.com/in/nathan-cartmell](https://www.linkedin.com/in/nathan-cartmell))

| Always happy to talk backend or telecoms.

ABOUT

I build the systems behind cloud communications software — provisioning, VoIP account management and billing for a UK platform serving over 200,000 users.

I started in technical support, moved into mobile development building the company's native iOS and Android clients, and now work primarily on backend services. Having dealt with customer problems directly and then built the interfaces those customers use, I have a broad view of the domain — and it shapes design decisions as much as the technical reasoning does.

EXPERIENCE

System Developer · Nebula

Dec 2023 – Present

- One of five engineers building a greenfield Kotlin microservices platform to replace a decade-old Java monolith — Spring Boot, gRPC, Protobuf and Cloud Spanner on GCP. Second in seniority on the project, owning the provisioning, VoIP and account domains.
- Schema and data ownership across both generations of the platform: designing entities and migrations for the new system, consolidating overlapping legacy models, and keeping the two consistent while they run side by side.
- Feature delivery end to end across the product range — customer portals, partner ordering, billing and provisioning — usually taking a requirement from specification through to release rather than building to someone else's design.
- Maintain the internal systems the company runs on, including the operations portal used daily by support, sales and provisioning staff.
- Act as effective third line for production issues — the escalation point when support and the wider development team cannot resolve something, diagnosing live against real customer impact.
- Platform-wide engineering work: OpenTelemetry tracing across every service and the gateway, static analysis and formatting standards, API documentation, and rebuilt test suites.

Mobile Developer · Nebula

Dec 2022 – Dec 2023

- Principal developer on the native iOS softphone in Swift, and its Android counterpart in Kotlin from late 2023 — maintained both alongside backend work until they were succeeded by a React Native rewrite in 2025.
- Built WebRTC video conferencing: in-call UI, waiting rooms, host admin controls, participant management and inbound conference joining.
- Delivered team chat with user tagging, markdown rendering, pinning, read state and SMS conversations — plus voicemail notifications, BLF presence indicators, call history and contact management.
- Shipped a single codebase as two separately branded white-label applications.

- Second-line support for the same communications platform I now build — diagnosing call quality, routing and provisioning issues across partner deployments.

SELECTED WORK

Provisioning platform migration

2026

- Provisioning is what configures a desk phone the moment it is plugged in — line keys, directories, dial plans, busy-lamp indicators — with nobody touching the handset. I rebuilt it from the legacy monolith into the new Kotlin platform and shipped it through staged releases without interrupting live provisioning.
- Supports Grandstream, Yealink, Cisco, Polycom, Fanvil and Gigaset hardware, each with its own firmware quirks. One fix meant reformatting responses to one XML tag per line after tracing a failure to handsets using fixed-size line-read buffers that silently truncated anything longer.
- Built the dual-write framework keeping legacy and new data models consistent throughout the migration — the seam the wider platform transition depends on.
- I had maintained the Python provisioning server it replaced for two years, so the rebuild carried forward behaviour I already owned rather than reverse-engineering someone else's. Device check-in intervals are randomised so that thousands of handsets do not poll simultaneously after a restart.

Virtual landline launch

2026

- GoLandline is a standalone brand selling a virtual landline to small businesses direct: its own storefront, checkout and dashboard, on the platform that otherwise sells through partners. I joined two weeks from a fixed launch date with a lot still outstanding, and ran the project as well as building on it — daily standups, daily progress to stakeholders, QA builds quick enough to test against.
- A good share of the work was deciding what not to build. The requirements had been drawn up months earlier and had drifted from what the platform could actually do — parts of the scope assumed behaviour we could not offer, and missed capability we already had. Call recording, for one, lost its own module and became an account flag that existed already.
- Built the internal side so support could answer for the product on day one: order search, a read-only subscription view, a tool to re-run a stalled provisioning order, a report of accounts whose billing and telephony state had drifted apart, and a handoff into the customer's own dashboard so an agent sees what they see.
- The price on a subscription is not what the customer pays: tax and discounts only exist once an invoice is drawn, so anything quoting a real figure has to ask Stripe to preview the invoice rather than read the price.

Order and event processing

2024 – Present

- The asynchronous backbone behind the platform: reseller and partner order processors, email subscribers, port verification handling, fraud alerting, and the record processor feeding call reporting.
- Spans both event-driven consumers on Pub/Sub and recurring scheduled work — including distributed locking so that a job fires once across every running instance rather than once per instance.
- Also covers number lifecycle — recycling deactivated numbers back into the available pool — and scheduled reconciliation against the payment and accounting systems.

Emergency address consolidation

2025

- UK operators must hold an accurate installation address for every number so that 999 calls reach the correct control room. Two overlapping address models had grown up in the database — number addresses and emergency addresses — leaving no single source of truth for a legally significant field.
- Consolidated onto one model and removed the legacy path entirely, reconciling four upstream carriers with differing address formats and validation rules, and adding validation at the point of entry rather than at

submission.

HOW I WORK

- I have reviewed over 600 pull requests across the organisation. A meaningful part of my contribution sits in other people's branches rather than my own — catching problems early is cheaper than catching them in production.
- I support developers joining the team, having come through the graduate route here myself and remembering what was unclear.
- Getting work out of the door often means driving the conversation rather than the code. I coordinate across QA, Product, Support, Marketing and Development to get a release agreed, tested and actually shipped.

TECHNICAL

LANGUAGES	Kotlin, Java, Python, Swift, TypeScript, SQL
JVM	Spring Boot, Spring Data, gRPC, Protobuf, Hibernate/JPA, Coroutines, OpenAPI, Spark Java
PYTHON	Flask, SQLAlchemy, Flask-Login, Jinja
DATA	Cloud Spanner, PostgreSQL, MySQL, Redis
PLATFORM	GCP, Pub/Sub, Docker, Unleash
OBSERVABILITY	OpenTelemetry, Micrometer, SonarQube
TESTING	JUnit, MockK, AssertJ, Testcontainers
MOBILE	Swift, Android, WebRTC
INTEGRATIONS	Stripe, GoCardless, Salesforce, Zendesk, Xero
DOMAIN	VoIP & SIP, Device provisioning, Number porting & lifecycle, Carrier integration, Emergency call routing, Ofcom compliance, Subscription billing, CDR & call reporting

CERTIFICATIONS

Cradlepoint Certified Network Expert

Ubiquiti Enterprise Wireless Admin

EDUCATION

Higher National Certificate, Computer Science · The Bedford College Group

2020 – 2021